

Proyecto

EVENTGLOBAL

ALUMNAS:



**BRENDA
LÓPEZ RAMOS**



**ANA CRISTINA
PEREZ LÓPEZ**



**BRITANI AGUILAR
LORENZANA**



**PROFESOR:
IVAN AZAMAR PALMA**

INDICE

INTRODUCCION	3
MARCO TEORICO	4
FRONTEND	7
FIREBASE	8
LOGIN	10
REGISTRO.....	12
AUTENTICACION	14
CONSUMO DE GOOOGLEMAPS	15
CONSUMO DE LA API WEATHER Y ONET PARA EVENTOS NATURALES Y CLIMA Y CHATGPT PARA RECOMENDACIONES	18
DESCRIPCION DE PERFIL	21
RESULTADOS PERFIL	23
BAKEND	24
RESULTADOS DE BAKEND	35
CONCLUSIONES	37

INTRODUCCION

EventGlobal es una innovadora aplicación web diseñada para proporcionar información en tiempo real sobre eventos naturales a nivel mundial. Esta aplicación integra diversas tecnologías y servicios para ofrecer una experiencia de usuario completa y eficiente. Utilizando la API de EONet de la NASA, EventGlobal permite a los usuarios conocer y monitorear eventos naturales como terremotos, incendios forestales, tormentas y más, proporcionando datos precisos y actualizados.

La implementación de Google Maps en EventGlobal permite la visualización geoespacial de estos eventos, ofreciendo una representación visual clara y detallada de su ubicación e impacto. Esta integración facilita a los usuarios la comprensión de la magnitud y alcance de los eventos naturales en diferentes regiones del mundo.

Una característica destacada de EventGlobal es la incorporación de inteligencia artificial a través de ChatGPT. Esta IA proporciona tips y recomendaciones útiles sobre cómo prepararse y reaccionar ante diversos eventos naturales, mejorando así la capacidad de los usuarios para enfrentar situaciones de emergencia con conocimiento y confianza.

EventGlobal está construida con una sólida arquitectura tecnológica que incluye Angular para el desarrollo del frontend. Se utiliza Angular Material para crear una interfaz de usuario moderna y responsiva, garantizando una experiencia de usuario atractiva y fácil de usar. En el backend, se emplea Node.js y Express para manejar las solicitudes y proporcionar un servicio eficiente y escalable. La base de datos de EventGlobal está gestionada con MongoDB Atlas, lo que asegura la integridad y disponibilidad de los datos en todo momento.

El objetivo principal de EventGlobal es proporcionar una herramienta útil y accesible para la comunidad global, ayudando a las personas a estar mejor informadas y preparadas ante los eventos naturales. Con la combinación de tecnologías avanzadas y servicios integrados, EventGlobal representa un ejemplo sobresaliente de cómo la tecnología puede ser utilizada para el bienestar y la seguridad de las personas en todo el mundo.

MARCO TEORICO

1. Arquitectura de aplicaciones web:

- Las aplicaciones web modernas se basan en una arquitectura cliente-servidor, donde el cliente (navegador web) solicita recursos y servicios al servidor a través de Internet.
- Esta arquitectura permite la construcción de aplicaciones accesibles desde cualquier dispositivo con conexión a Internet y facilita la distribución de la lógica de la aplicación entre el cliente y el servidor.

2. Node.js y Express:

- Node.js es un entorno de ejecución de JavaScript del lado del servidor que permite desarrollar aplicaciones web escalables y de alto rendimiento.
- Express es un marco web para Node.js que simplifica la creación de servidores web y APIs mediante el enrutamiento de solicitudes HTTP y la gestión de middleware.

3. API REST:

- Las API RESTful (Representational State Transfer) son una arquitectura de diseño de software que utiliza el protocolo HTTP para la creación de servicios web.
- Las API REST permiten la comunicación entre clientes y servidores mediante operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre recursos que se representan de manera uniforme.

4. Seguridad en aplicaciones web:

- La seguridad es un aspecto fundamental en el desarrollo de aplicaciones web, especialmente en el ámbito del comercio electrónico donde se manejan datos sensibles de los usuarios.
- Se utilizan técnicas como el cifrado de contraseñas, la validación de datos de entrada, la autenticación de usuarios y la protección contra ataques comunes como la inyección de SQL y la denegación de servicio.

5. Bases de datos NoSQL y MongoDB:

- Las bases de datos NoSQL ofrecen una alternativa flexible y escalable a las bases de datos relacionales tradicionales, especialmente en entornos donde se requiere una alta disponibilidad y rendimiento.
- MongoDB es una base de datos NoSQL de código abierto que utiliza un modelo de documentos JSON para almacenar datos, lo que facilita la integración con aplicaciones web desarrolladas en Node.js.

6. Librerías y herramientas adicionales:

- En el desarrollo de aplicaciones web, se utilizan diversas librerías y herramientas para simplificar tareas comunes como el manejo de datos, la autenticación de usuarios, la manipulación de fechas y la gestión de archivos.
- Ejemplos de estas herramientas incluyen bcrypt para el cifrado de contraseñas, body-parser para el análisis de datos en formato JSON, jwt-simple para la autenticación basada en tokens JWT, entre otras.

7. Ventajas de usar Mongoose:

- Mongoose es una biblioteca de modelado de objetos MongoDB para Node.js que proporciona una capa de abstracción sobre las operaciones de base de datos.
- Otra ventaja clave de Mongoose es su soporte para middleware, que permite ejecutar funciones antes o después de realizar operaciones en la base de datos.

8. Visual Studio Code Visual Studio Code (VS Code)

Es un editor de código fuente desarrollado por Microsoft para Windows, macOS y Linux. Popular entre desarrolladores, destaca por su ligereza, rapidez y extensibilidad. Ofrece características como un editor de texto avanzado con sintaxis resaltada y autocompletado, integración de Git para control de versiones, depuración de código, terminal integrada y un vasto ecosistema de extensiones que amplían su funcionalidad.

9. Librerías API (Interfaces de Programación de Aplicaciones)

Permiten la comunicación entre sistemas de software mediante un conjunto de definiciones y protocolos. Las librerías API son colecciones de funciones y métodos que facilitan la interacción con una API, proporcionando abstracción y consistencia. Estas librerías simplifican tareas comunes, mejoran la productividad y reducen la necesidad de conocer los detalles internos de la implementación.

10. Dependencias

Son componentes externos que un proyecto necesita para funcionar correctamente, como librerías o frameworks. Gestionarlas eficientemente es crucial para la estabilidad y escalabilidad del software. Herramientas como npm (Node.js), pip (Python) y Maven (Java) facilitan la gestión de dependencias. Es vital especificar versiones para garantizar compatibilidad y seguridad, y mantenerlas actualizadas para mitigar vulnerabilidades.

- REST es un estilo arquitectónico para servicios web escalables, basado en principios como:
- Recursos: Identificados por URL únicas.

- Métodos HTTP: Utiliza GET, POST, PUT y DELETE para operaciones de lectura, creación, actualización y eliminación de recursos.
- Representaciones: Los recursos pueden tener múltiples representaciones (JSON, XML).
- Stateless: Cada solicitud contiene toda la información necesaria, sin almacenar estado en el servidor.
- Cacheable: Las respuestas pueden ser cacheadas para mejorar el rendimiento.
- Interfaz uniforme: Simplifica la arquitectura y permite la evolución independiente de sus componentes.

Visual Studio Code, junto con la gestión adecuada de librerías API y dependencias, proporciona un entorno de desarrollo robusto y eficiente. REST, como estilo arquitectónico, facilita la creación de servicios web escalables y mantenibles, promoviendo la interoperabilidad y la evolución independiente de los sistemas. 3

Correr el demonio de MongoDB" se refiere a iniciar el servicio principal de MongoDB, que es el programa que maneja la base de datos. En terminología de sistemas operativos y bases de datos, un "demonio" (o "daemon" en inglés) es un programa que se ejecuta en segundo plano y realiza tareas específicas, como manejar solicitudes de base de datos.

1. Demonio de MongoDB: El demonio de MongoDB se llama ``mongod``. Este proceso es responsable de manejar las solicitudes de la base de datos, gestionar los datos almacenados y llevar a cabo operaciones como lectura, escritura y actualización de datos.
2. Inicio del Demonio: Para iniciar el demonio de MongoDB, generalmente se usa el comando ``mongod`` en la línea de comandos o terminal del sistema operativo. Dependiendo de la configuración, puede ser necesario especificar opciones adicionales, como la ubicación del archivo de configuración o el directorio de datos.
3. Configuración y Datos: Al iniciar, ``mongod`` puede tomar varias opciones de configuración, incluyendo la ubicación de los datos (``--dbpath``), el archivo de configuración (``--config``), y parámetros de red y seguridad.
4. Importancia de Correr el Demonio de MongoDB

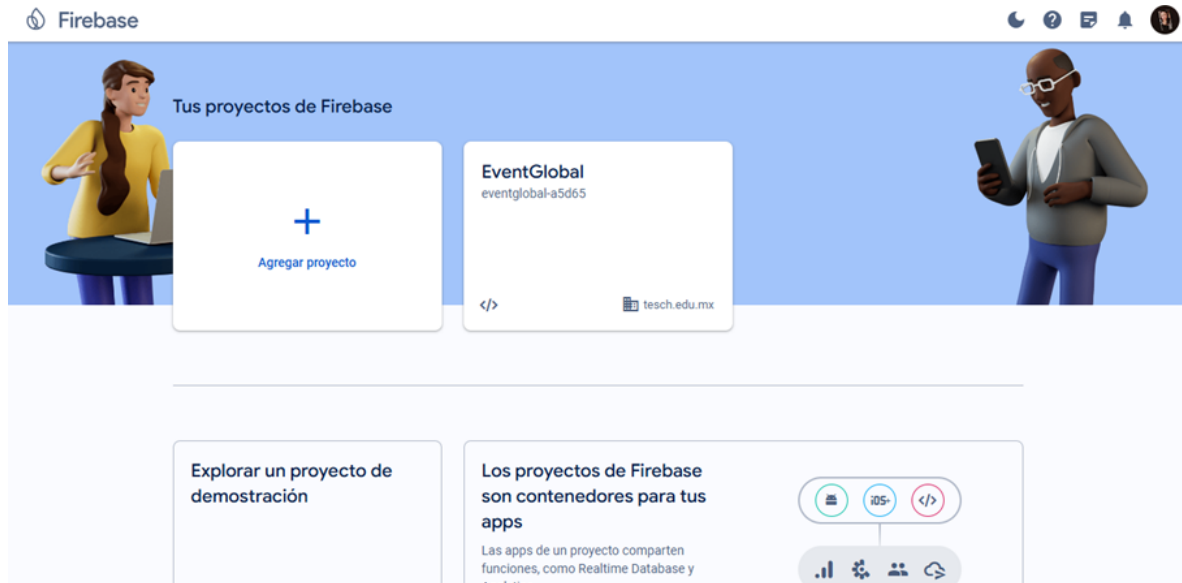
Disponibilidad de la Base de Datos: ``mongod`` debe estar corriendo para que las aplicaciones puedan conectarse a la base de datos y realizar operaciones. Gestión de Conexiones: Maneja todas las conexiones entrantes de los clientes y gestiona las operaciones de lectura y escritura en la base de datos.

DESARROLLO

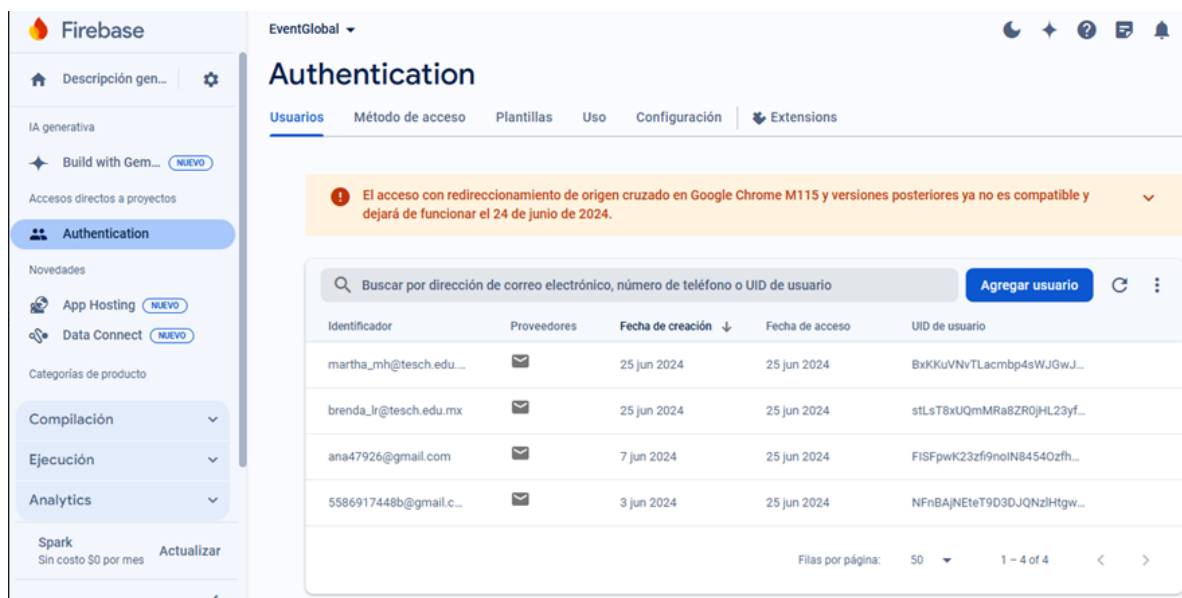
FRONTEND

Firestore

En esta imagen se muestra el panel principal de Firebase donde se gestionan los proyectos. Se observa el proyecto actual llamado "EventGlobal" y la opción de agregar nuevos proyectos. Este panel permite navegar y acceder rápidamente a las configuraciones y servicios asociados a cada proyecto.

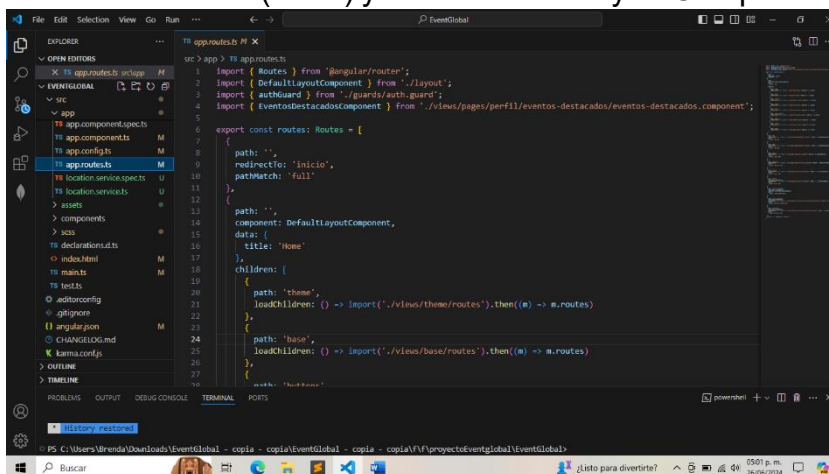


Esta imagen muestra la sección de "Authentication" del proyecto "EventGlobal". Aquí se listan los usuarios registrados junto con sus detalles como el proveedor de autenticación, fecha de creación y UID. También se puede agregar nuevos usuarios manualmente. Es crucial para gestionar y monitorear el acceso de los usuarios a la aplicación.



App.routes

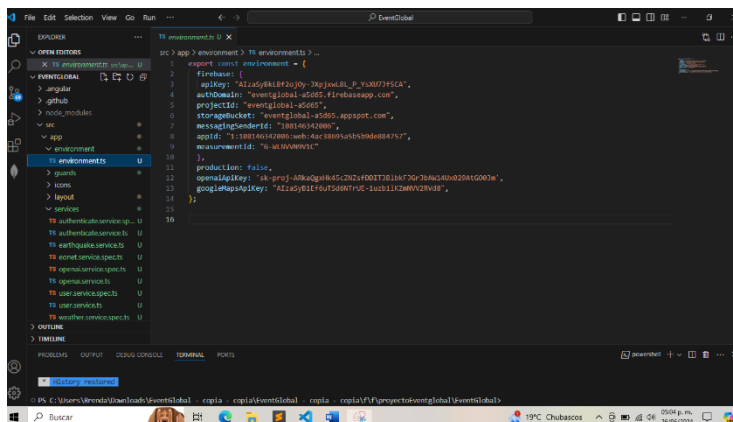
El archivo `app.routes.ts` en un proyecto Angular define las rutas de navegación de la aplicación. Redirige la ruta base al inicio (inicio) y utiliza `DefaultLayoutComponent` para la estructura principal, cargando módulos hijos (como `theme`, `base`, `buttons`, etc.) de forma diferida. También configura rutas específicas para páginas de error (404 y 500), autenticación (login y register), y varias rutas protegidas por un guard (`authGuard`) para componentes como perfil, eventos-destacados, historial-eventos y ubicacion. La ruta comodín redirige cualquier ruta no definida al dashboard.



9

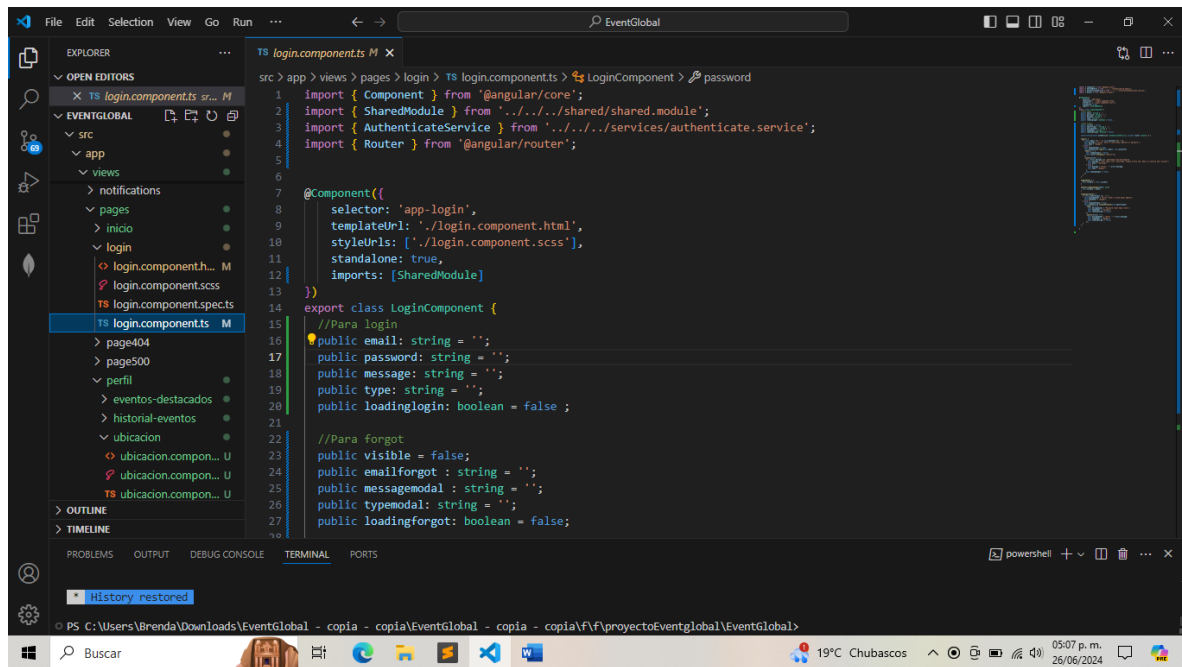
environment.ts

El archivo `environment.ts` en una aplicación Angular contiene la configuración específica para el entorno de desarrollo. Incluye las credenciales de Firebase necesarias para inicializar servicios como autenticación, almacenamiento, y Firestore, junto con la clave de API de Google Maps y la clave de API de OpenAI. El objeto `firebase` contiene detalles como `apiKey`, `authDomain`, `projectId`, `storageBucket`, `messagingSenderId`, `appId` y `measurementId`. La propiedad `production` se establece en `false` para indicar que es un entorno de desarrollo.



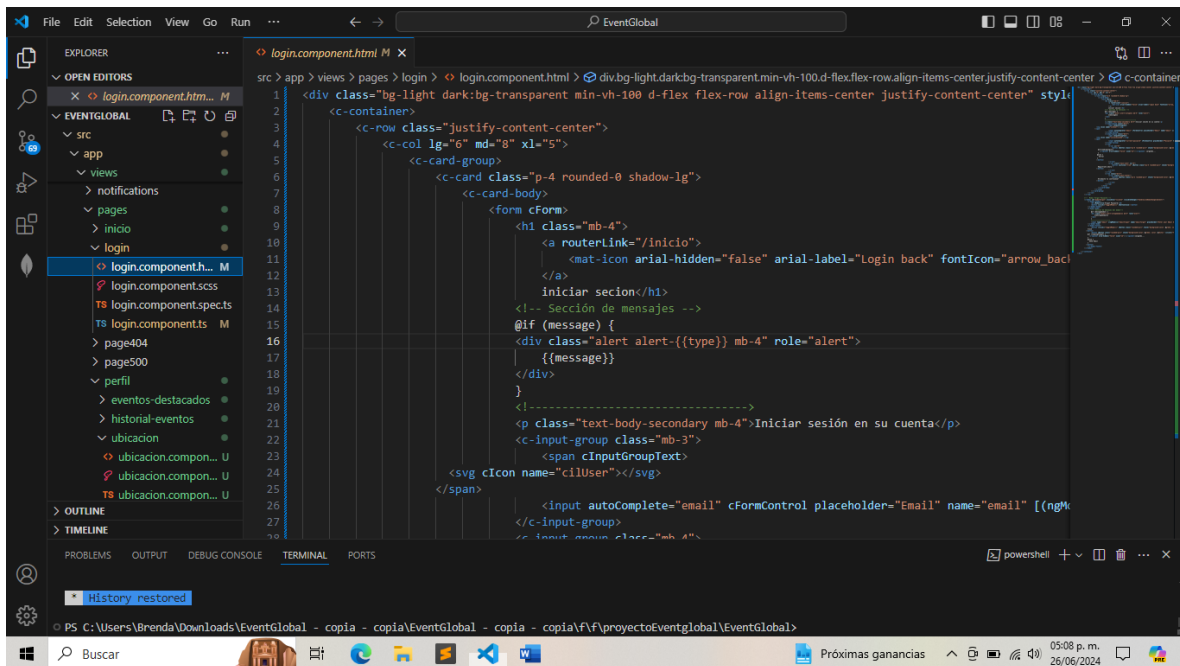
Login.component.ts

El archivo login.component.ts define el componente de inicio de sesión en una aplicación Angular. Gestiona el formulario de inicio de sesión, validando la entrada del usuario y utilizando AuthenticateService para autenticar con Firebase. Si la autenticación es exitosa, redirige al usuario a la página de perfil; de lo contrario, muestra mensajes de error. También incluye funcionalidad para la recuperación de contraseña, mostrando un modal y enviando un correo de restablecimiento si el usuario proporciona un email válido. Además, maneja estados de carga y mensajes de feedback para ambas funcionalidades.

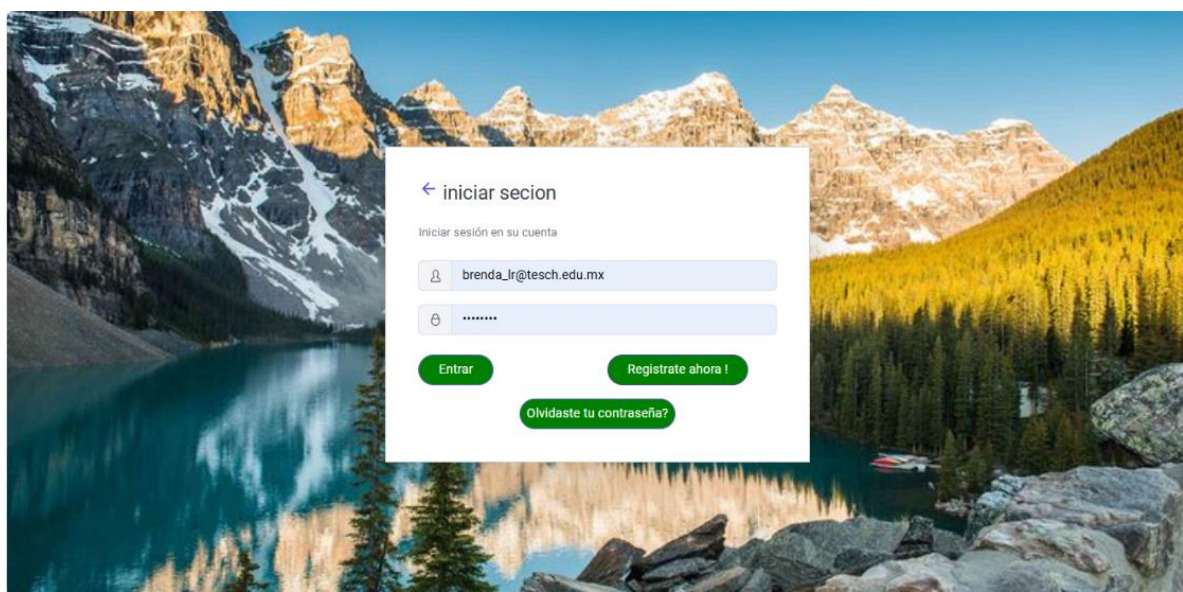


Login.component.html

El archivo login.component.html define la interfaz del componente de inicio de sesión en una aplicación Angular. Utiliza un fondo personalizado y contiene un formulario de inicio de sesión que permite a los usuarios ingresar su correo electrónico y contraseña, con un botón para enviar los datos. Si el inicio de sesión falla, se muestran mensajes de error. También incluye enlaces para registrar una nueva cuenta y recuperar contraseñas olvidadas, mostrando un modal para la recuperación de contraseña cuando es necesario. Los estados de carga y mensajes de feedback proporcionan una mejor experiencia de usuario.



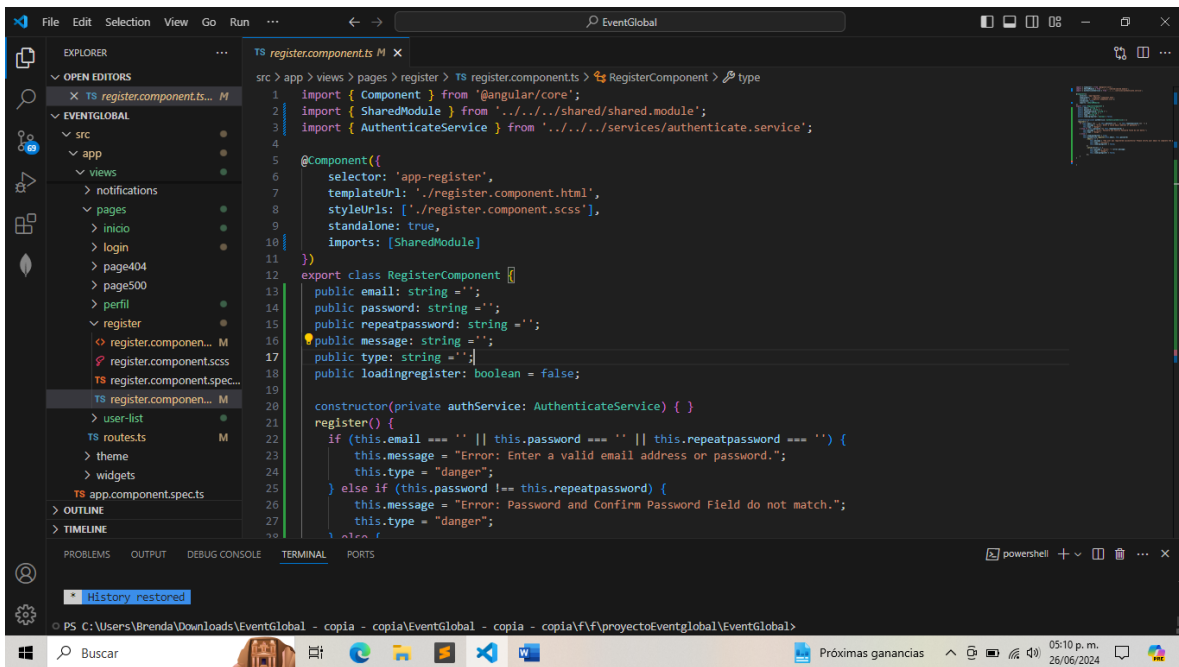
RESULTADOS LOGIN



El resultado del componente de inicio de sesión (login.component.html) es una interfaz de usuario donde los usuarios pueden ingresar su correo electrónico y contraseña para acceder a sus cuentas. La interfaz incluye campos para email y contraseña, un botón para iniciar sesión que muestra un indicador de carga mientras se procesa la autenticación, y enlaces para registrarse o recuperar la contraseña si es necesario. El fondo de la página muestra una imagen de un paisaje montañoso, proporcionando una apariencia visualmente agradable y atractiva para los usuarios.

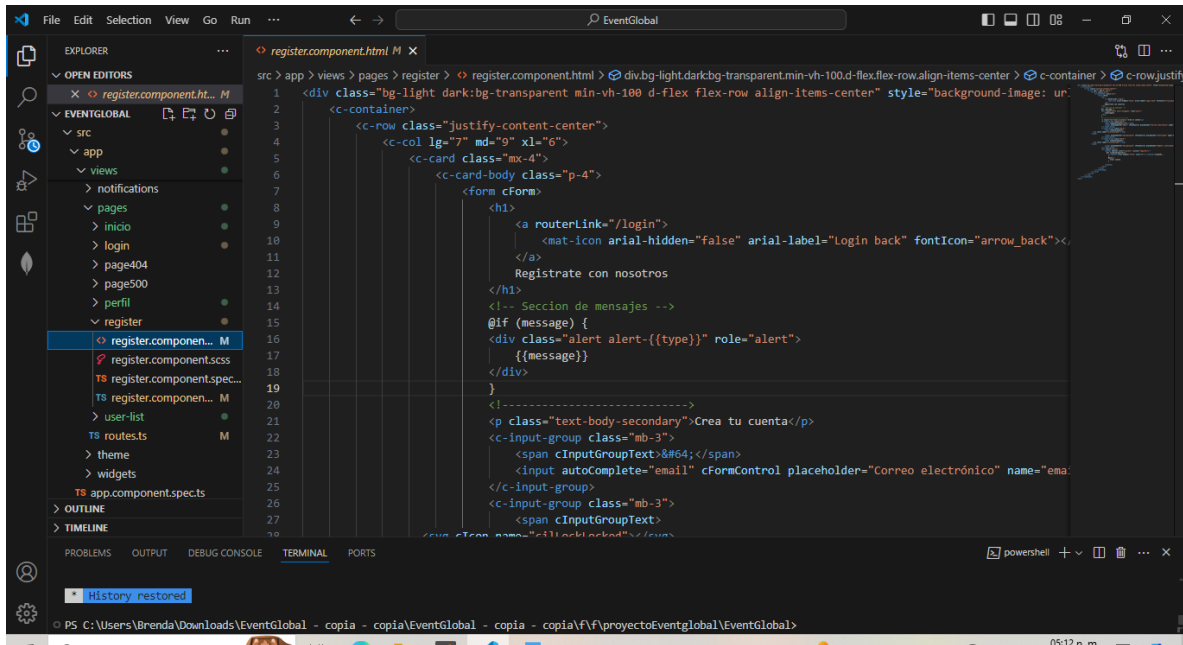
Register.component.ts

El archivo `register.component.ts` define el componente de registro de usuarios en una aplicación Angular. Proporciona un formulario donde los usuarios pueden ingresar su correo electrónico, contraseña y repetir la contraseña para crear una nueva cuenta. El componente valida que los campos no estén vacíos y que las contraseñas coincidan. Si las validaciones son correctas, utiliza `AuthenticateService` para registrar al usuario y muestra mensajes de éxito o error según corresponda. También gestiona un estado de carga mientras se procesa el registro para mejorar la experiencia del usuario.



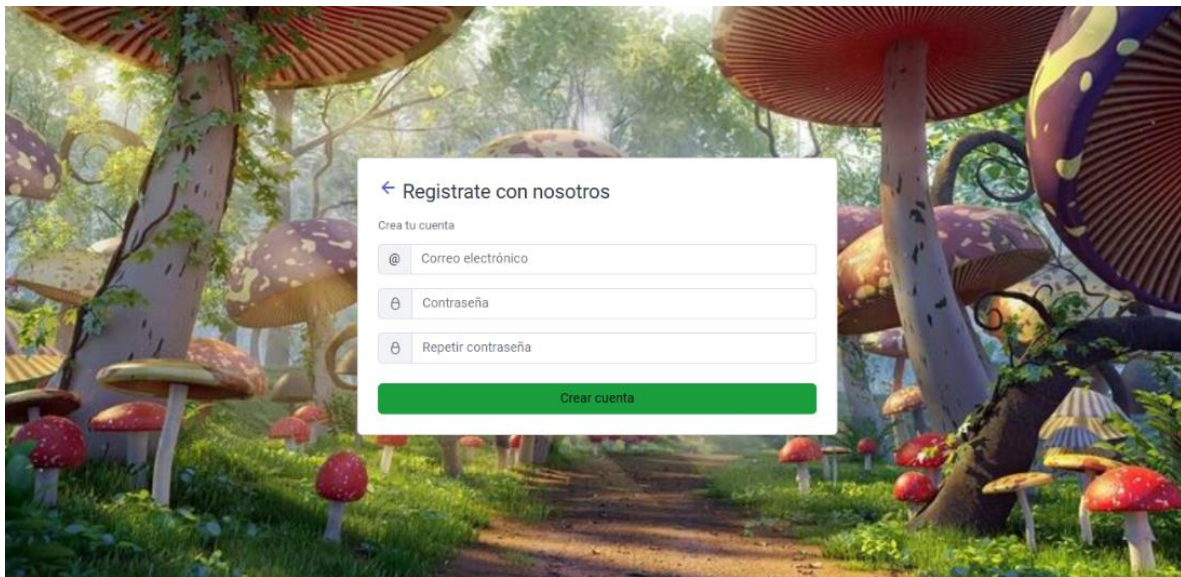
Register.component.html

El archivo `register.component.html` define la interfaz de usuario para el registro de nuevos usuarios en una aplicación Angular. Presenta un formulario con campos para el correo electrónico, contraseña y repetir la contraseña, acompañado de un botón para crear la cuenta. Incluye una sección para mostrar mensajes de error o éxito. La interfaz está diseñada sobre un fondo de imagen atractivo, alineada al centro de la pantalla, y contiene un enlace para volver a la página de inicio de sesión. Además, el botón de registro muestra un indicador de carga mientras se procesa la solicitud.



```
src > app > views > pages > register > register.component.html > div.bg-light.darkbg-transparent.min-vh-100.d-flex.flex-row.align-items-center > c-container > c-row.justify-content-center
1 <div class="bg-light darkbg-transparent min-vh-100 d-flex flex-row align-items-center" style="background-image: url('...');">
2 <c-container>
3 <c-row class="justify-content-center">
4 <c-col lg="7" md="9" xl="6">
5 <c-card class="mx-4">
6 <c-card-body class="p-4">
7 <form cForm>
8 <h1>
9 <a routerLink="/login">
10 <mat-icon arial-hidden="false" arial-label="Login back" fontIcon="arrow_back"></mat-icon>
11 </a>
12 Registrare con nosotros
13 </h1>
14 <!-- Seccion de mensajes -->
15 @if (message) {
16 <div class="alert alert-{{type}}" role="alert">
17 {{message}}
18 </div>
19 }
20 <!-- Seccion de creacion de cuenta -->
21 <p class="text-body-secondary">Crea tu cuenta</p>
22 <c-input-group class="mb-3">
23 <span cInputGroupText:#64;</span>
24 <input autoComplete="email" cFormControl placeholder="Correo electrónico" name="ema
25 </c-input-group>
26 <c-input-group class="mb-3">
27 <span cInputGroupText:
28 </c-input-group>
29 <input type="password" cFormControl placeholder="Contraseña" name="password">
30 </c-input-group>
31 <c-input-group class="mb-3">
32 <span cInputGroupText:
33 </c-input-group>
34 <input type="password" cFormControl placeholder="Repetir contraseña" name="password2">
35 </c-input-group>
36 <button type="submit" class="btn btn-primary w-100">Crear cuenta</button>
37 </c-card-body>
38 </c-card>
39 </c-col>
40 </c-row>
41 </c-container>
42 </div>
```

Resultados registro

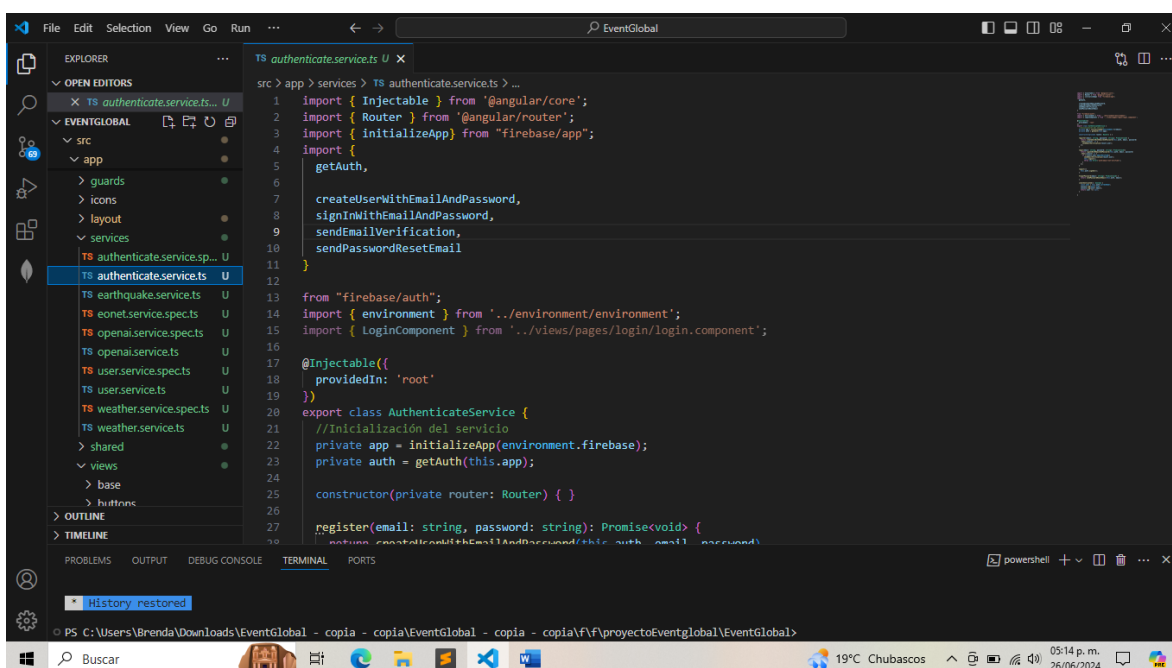


El resultado del componente de registro es una interfaz de usuario limpia y atractiva para la creación de cuentas nuevas en la aplicación. Presenta un formulario con campos para ingresar el correo electrónico, la contraseña y la confirmación de la contraseña, acompañado de un botón verde claro para "Crear cuenta". La interfaz está ubicada sobre un fondo visual de hongos en un entorno natural, proporcionando una apariencia visualmente atractiva. También incluye un enlace para volver a la página de inicio de sesión, mejorando la navegación del usuario.

Authenticate.service

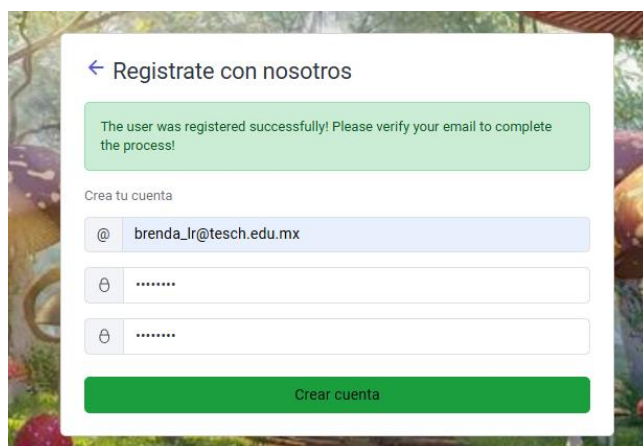
El archivo `authenticate.service.ts` define un servicio en Angular para manejar la autenticación de usuarios mediante Firebase. Inicializa la aplicación Firebase y la instancia de autenticación. Proporciona métodos para registrar usuarios (`register`), iniciar sesión (`login`), enviar correos de verificación, restablecer contraseñas (`forgotPassword`), cerrar sesión (`logout`), y verificar si un usuario está autenticado (`isAuthenticated`). Si un usuario intenta iniciar sesión sin haber verificado su correo electrónico, el servicio envía un correo de verificación y fuerza la salida del usuario. Este servicio centraliza la lógica de autenticación y facilita su gestión en toda la aplicación.

14



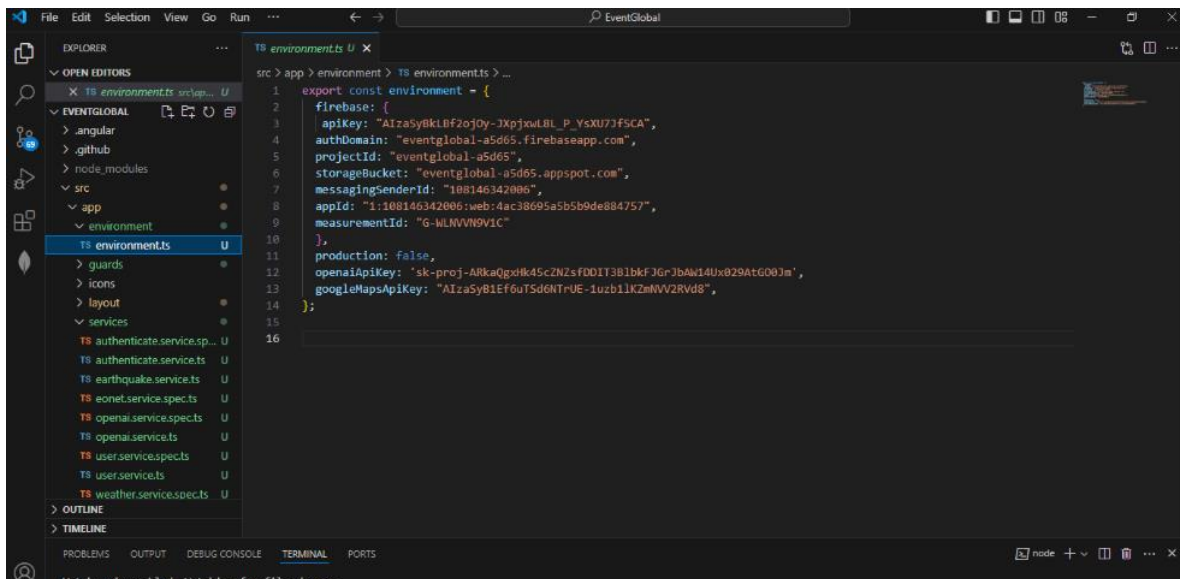
Resultados de autenticación

Después de registrarte, apareció un mensaje en verde indicando que el registro fue exitoso. El texto del mensaje dice: "The user was registered successfully! Please verify your email to complete the process!" Este mensaje informa que el usuario ha sido registrado con éxito y solicita que verifique su correo electrónico para completar el proceso de registro.



Consumo de la api googlemaps

La imagen muestra el archivo `environment.ts` de un proyecto Angular, que configura el entorno de desarrollo con claves y configuraciones necesarias para consumir diversas APIs. Específicamente, contiene las credenciales de Firebase (como `apiKey`, `authDomain`, `projectId`, etc.), que permiten la integración con servicios de autenticación, almacenamiento y bases de datos en tiempo real de Firebase. Además, incluye una clave de API de Google Maps (`googleMapsApiKey`) para servicios de mapas y una clave de API de OpenAI (`openaiApiKey`) para consumir servicios de inteligencia artificial. La propiedad `production` está configurada como `false`, indicando que es un entorno de desarrollo.



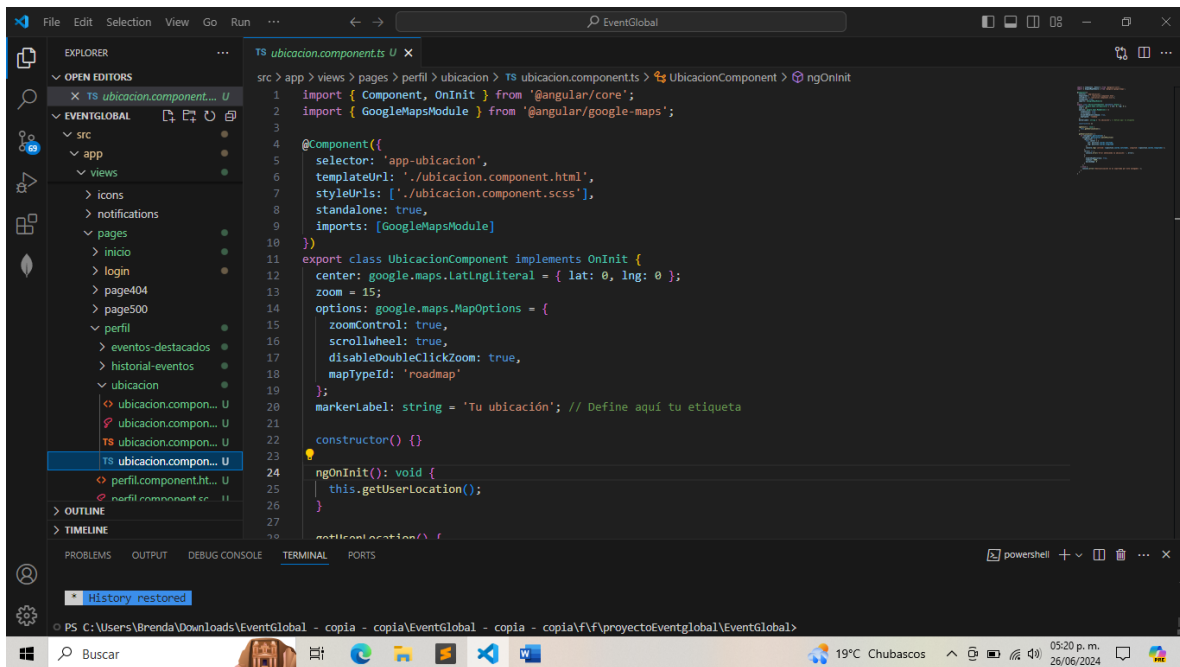
```

src > app > environment > TS environments > ...
1  export const environment = {
2    firebase: {
3      apiKey: "AIzaSyBklBf2oJ0y-JXpJxwLBI_P_YsXU7JfSCA",
4      authDomain: "eventglobal-a5d65.firebaseio.com",
5      projectId: "eventglobal-a5d65",
6      storageBucket: "eventglobal-a5d65.appspot.com",
7      messagingSenderId: "108146342006",
8      appId: "1:108146342006:web:4ac38695a5b5b9de884757",
9      measurementId: "G-WLVVW9V1C"
10   },
11   production: false,
12   openaiApiKey: 'sk-proj-ARkaQgdHk45cZNzfDDIT301bkfJGrJbAw14Ux029atG00jm',
13   googleMapsApiKey: "AIzaSyB1Ef6uTSdGnTrUE-1uzb1JKZmNVV2Rvd8",
14 };
15
16

```

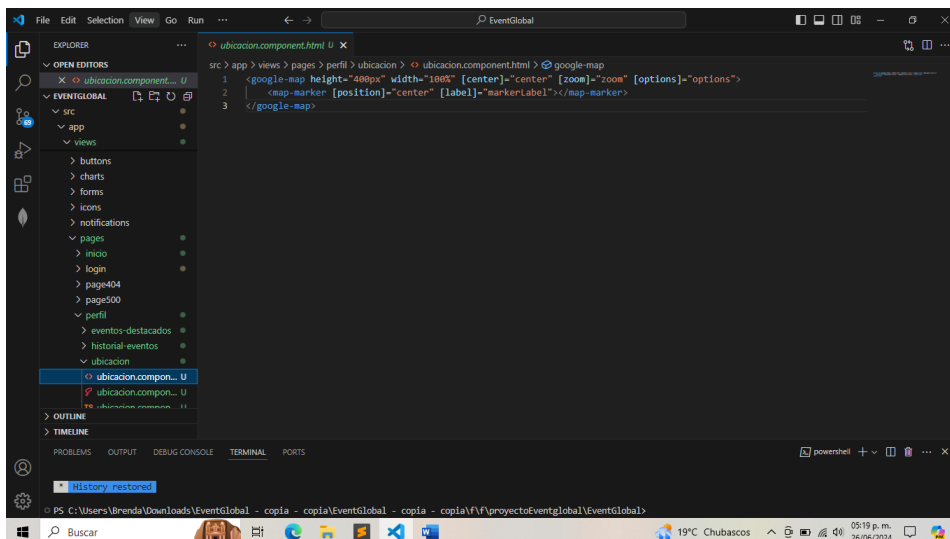
Ubicación.component.ts

El archivo `ubicacion.component.ts` en un proyecto Angular define un componente para gestionar y mostrar ubicaciones usando Google Maps. Este componente se integra con la API de Google Maps mediante la clave de API proporcionada en `environment.ts`. Utiliza servicios de Google Maps para cargar y mostrar un mapa interactivo, permitir a los usuarios buscar ubicaciones y posiblemente marcar o guardar estas ubicaciones en la base de datos de Firebase. La interacción incluye inicializar el mapa, manejar eventos de usuario como clics en el mapa, y actualizar la vista según la interacción del usuario.

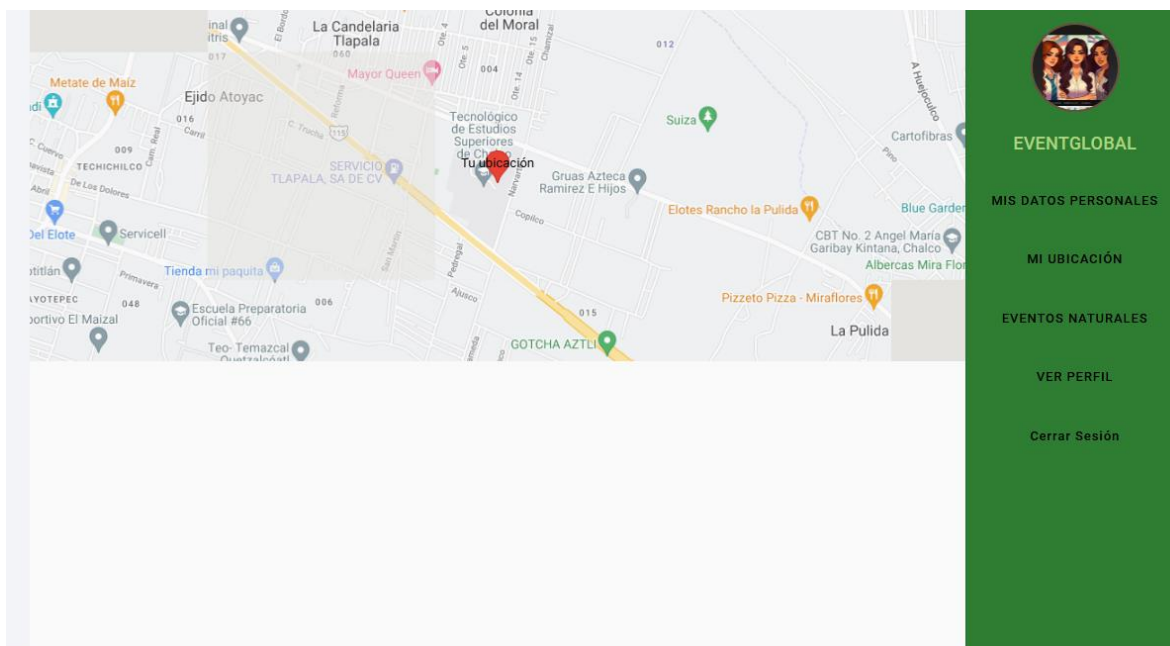


Ubicación.component.html

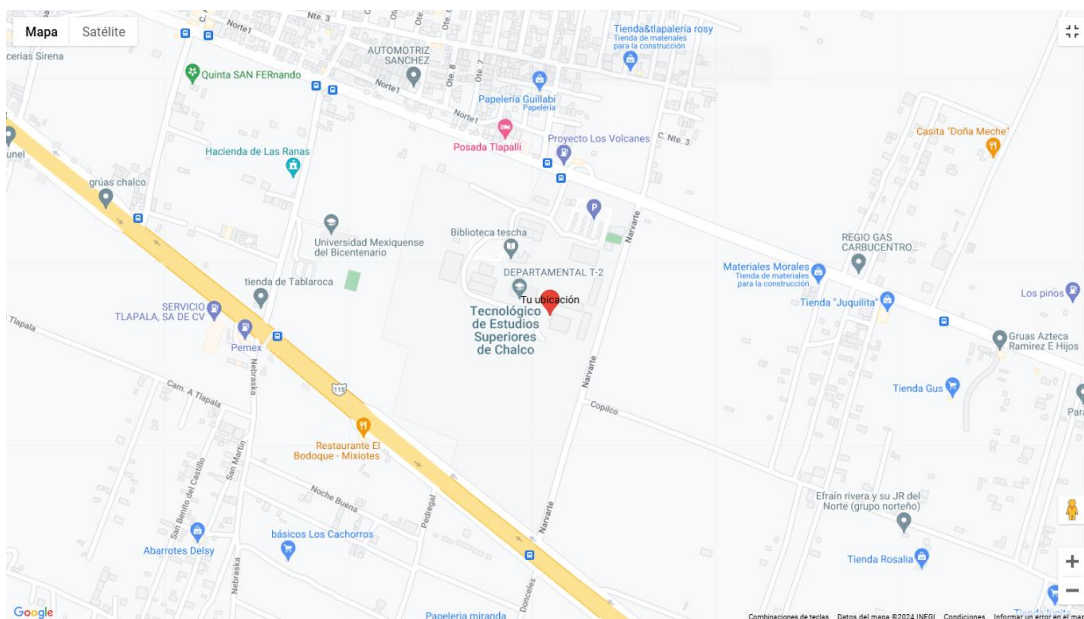
El archivo `ubicacion.component.html` define la estructura HTML para el componente de ubicación utilizando Google Maps en una aplicación Angular. El código muestra un mapa de Google con un marcador central. Específicamente, utiliza el elemento `<google-map>` con atributos como `height`, `width`, `center`, `zoom`, y `options` para configurar el mapa. Dentro del mapa, un elemento `<map-marker>` posiciona un marcador en el centro, con una etiqueta opcional. Este componente permite a los usuarios interactuar con el mapa, proporcionando una visualización geográfica en la aplicación.



Resultado de ubicación con googlemaps



La imagen muestra el resultado del componente de ubicación utilizando Google Maps en la aplicación "EventGlobal". En la pantalla se visualiza un mapa centrado en una ubicación específica, con un marcador indicando el "Tecnológico de Estudios Superiores de Tianguistenco". A la derecha, un menú de navegación vertical permite acceder a diferentes secciones de la aplicación, como "Mis Datos Personales", "Mi Ubicación", "Eventos Naturales", "Ver Perfil" y "Cerrar Sesión". Esta interfaz proporciona una experiencia interactiva y visualmente organizada para la gestión de ubicaciones dentro de la aplicación.



Consumo de la api weather y onet para eventos naturales y clima y chatgpt para recomendaciones

weather.service.spec.ts

```
TS weather.service.spec.ts U X
src > app > services > TS weather.service.spec.ts > ...
1 import { TestBed } from '@angular/core/testing';
2
3 import { WeatherService } from '../weather.service';
4
5 describe('WeatherService', () => {
6   let service: WeatherService;
7
8   beforeEach(() => {
9     TestBed.configureTestingModule({});
10    service = TestBed.inject(WeatherService);
11  });
12
13  it('should be created', () => {
14    expect(service).toBeTruthy();
15  });
16 });
17
```

El archivo `weather.service.spec.ts` configura pruebas para el servicio `WeatherService` en una aplicación Angular. Este servicio se encarga de consumir una API de clima para obtener datos meteorológicos. En el bloque de pruebas, se utiliza `TestBed` para configurar un entorno de prueba e inyectar una instancia del `WeatherService`. El caso de prueba básico verifica que el servicio se crea correctamente. El

`WeatherService` interactúa con una API de clima (no mostrada en el código) para proporcionar funcionalidades como obtener condiciones climáticas actuales o pronósticos para una ubicación específica.

eonet.service.spec.ts

```
TS eonet.service.spec.ts U X
src > app > services > TS eonet.service.spec.ts > ...
1 import { TestBed } from '@angular/core/testing';
2
3 import { EonetService } from '../eonet.service';
4
5 describe('EonetService', () => {
6   let service: EonetService;
7
8   beforeEach(() => {
9     TestBed.configureTestingModule({});
10    service = TestBed.inject(EonetService);
11  });
12
13  it('should be created', () => {
14    expect(service).toBeTruthy();
15  });
16 });
17
```

El archivo `eonet.service.spec.ts` configura pruebas para el `EonetService`, el cual consume la API de la Earth Observatory Natural Event Tracker (EONET) para obtener información sobre eventos naturales. Utiliza `TestBed` para configurar el entorno de prueba e inyectar una instancia de `EonetService`. La prueba básica verifica que el servicio se crea correctamente. El `EonetService` se encarga de realizar solicitudes a la API de EONET para obtener datos de eventos naturales, como incendios, tormentas y terremotos, proporcionando estos datos a la aplicación para su uso en diversas funcionalidades.

openai.service.ts

```
TS openai.service.ts U x
src > app > services > TS openai.service.ts > OpenAiService > getRecommendations
1 // src/app/services/openai.service.ts
2 import { Injectable } from '@angular/core';
3 import { HttpClient, HttpHeaders } from '@angular/common/http';
4 import { Observable } from 'rxjs';
5 import { environment } from '../environment/environment';
6
7 @Injectable({
8   providedIn: 'root',
9 })
10 export class OpenAiService {
11   private apiUrl = 'https://api.openai.com/v1/chat/completions';
12   private apiKey = environment.openaiApiKey;
13
14   constructor(private http: HttpClient) {}
15
16   getRecommendations(prompt: string): Observable<any> {
17     const headers = new HttpHeaders({
18       'Content-Type': 'application/json',
19       'Authorization': `Bearer ${this.apiKey}`,
20     });
21
22     const body = {
23       model: 'gpt-3.5-turbo',
24       messages: [{ role: 'user', content: prompt }],
25       max_tokens: 150,
26       n: 1,
27       stop: null,
28       temperature: 0.7,
29     };
30   }
```

El archivo openai.service.ts en la aplicación Angular define un servicio para consumir la API de OpenAI con el propósito de obtener recomendaciones relacionadas con eventos naturales. El servicio, OpenAiService, utiliza HttpClient para realizar solicitudes HTTP a la API de OpenAI. La función getRecommendations acepta un prompt como entrada, construye los encabezados HTTP con la clave de API obtenida de environment.ts, y envía una solicitud POST con el cuerpo que especifica el modelo gpt-3.5-turbo, los mensajes, y otros parámetros

de configuración como max_tokens y temperature. El servicio devuelve un Observable que permite a otros componentes suscribirse y recibir las recomendaciones generadas por el modelo de OpenAI.

Resultados de eventos naturales con todas las apis



Mapa Satellite

☐ Relieve

Combinaciones de teclas | Datos del mapa ©2024 | Condiciones

Pronóstico del Tiempo para Mexico City, Mexico

2024-06-26

Temperatura: 17.9°C

Condición: Heavy rain



Probabilidad de lluvia: 98%

2024-06-27



EVENTGLOBAL

MIS DATOS PERSONALES

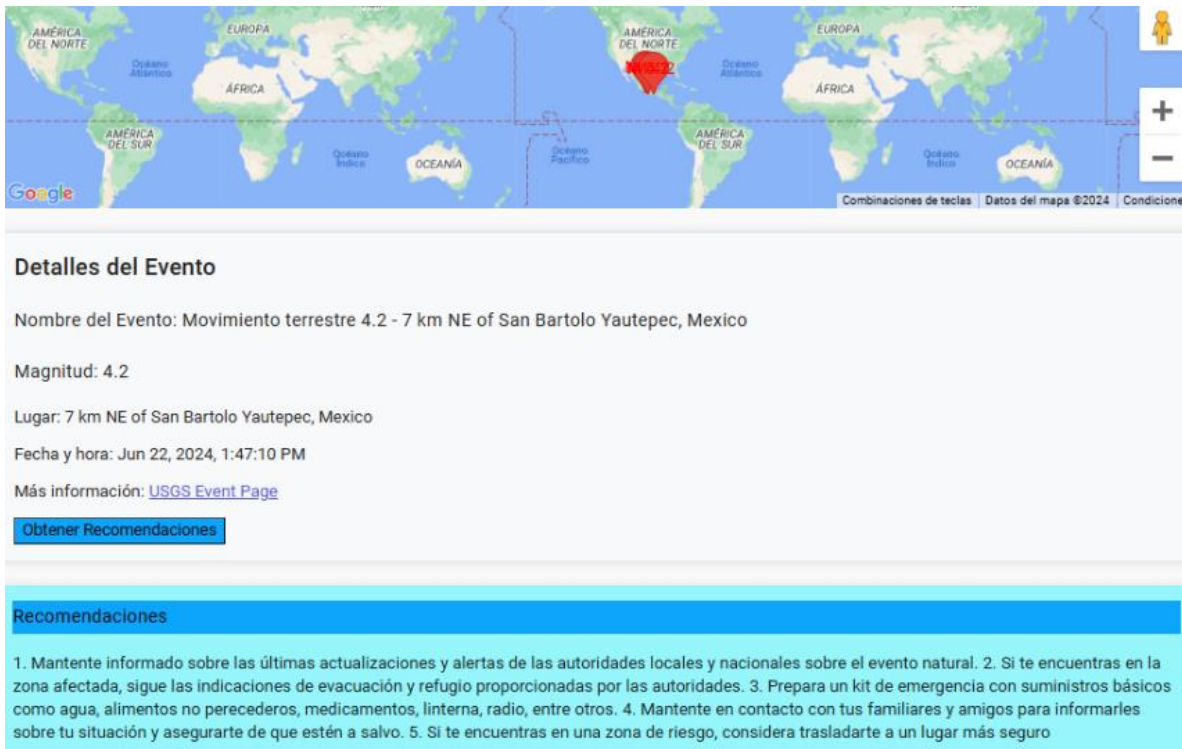
MI UBICACIÓN

EVENTOS NATURALES

VER PERFIL

Cerrar Sesión

Esta imagen muestra el pronóstico del tiempo para la Ciudad de México, México, integrado en la aplicación "EventGlobal". En la parte superior se ve un mapa de Google Maps que destaca la ubicación. Debajo del mapa, se presentan detalles del pronóstico del clima para el 26 de junio de 2024, incluyendo temperatura, condiciones climáticas y la probabilidad de lluvia. La interfaz proporciona una vista clara y detallada del clima, mejorando la planificación y la preparación de los usuarios.



Detalles del Evento

Nombre del Evento: Movimiento terrestre 4.2 - 7 km NE of San Bartolo Yautepec, Mexico

Magnitud: 4.2

Lugar: 7 km NE of San Bartolo Yautepec, Mexico

Fecha y hora: Jun 22, 2024, 1:47:10 PM

Más información: [USGS Event Page](#)

[Obtener Recomendaciones](#)

Recomendaciones

1. Mantente informado sobre las últimas actualizaciones y alertas de las autoridades locales y nacionales sobre el evento natural. 2. Si te encuentras en la zona afectada, sigue las indicaciones de evacuación y refugio proporcionadas por las autoridades. 3. Prepara un kit de emergencia con suministros básicos como agua, alimentos no perecederos, medicamentos, linterna, radio, entre otros. 4. Mantente en contacto con tus familiares y amigos para informarles sobre tu situación y asegurarte de que estén a salvo. 5. Si te encuentras en una zona de riesgo, considera trasladarte a un lugar más seguro

Esta imagen presenta información detallada sobre un evento natural, específicamente un terremoto. El evento se describe con detalles como la magnitud, la ubicación y la fecha y hora del suceso. También hay un botón para obtener recomendaciones, que al ser presionado, muestra consejos generados por la IA de OpenAI para manejar la situación del desastre. El uso de Google Maps para mostrar la ubicación y las recomendaciones en texto mejora significativamente la utilidad y la experiencia del usuario al proporcionar información y acciones relevantes.

DESCRIPCION DEL PERFIL EN GENERAL

Perfil.component.ts

El archivo perfil.component.ts define el componente de perfil en una aplicación Angular, que gestiona la visualización y edición de la información del usuario, incluyendo datos personales, ubicación y eventos naturales destacados. Este componente utiliza varios módulos de Angular Material como MatButtonModule, MatFormFieldModule, MatInputModule, y MatSidenavModule para crear una interfaz de usuario interactiva. Importa y utiliza componentes como UbicacionComponent y EventosDestacadosComponent para integrar funcionalidades relacionadas con la ubicación y eventos. Además, UserService se utiliza para realizar operaciones CRUD sobre los datos del usuario, como creación, obtención, actualización y eliminación. El componente maneja formularios reactivos para la entrada y validación de datos del usuario y permite cambiar entre diferentes secciones, incluyendo la opción de cerrar sesión.

21

```
TS perfil.component.ts U x
src > app > views > pages > perfil > TS perfil.component.ts > PerfilComponent
1  import { Component } from '@angular/core';
2  import { MatButtonModule } from '@angular/material/button';
3  import { MatFormFieldModule } from '@angular/material/form-field';
4  import { MatInputModule } from '@angular/material/input';
5  import { CommonModule } from '@angular/common';
6  import { RouterModule, Router } from '@angular/router';
7  import { MatSidenavModule } from '@angular/material/sidenav';
8  import { NgIf } from '@angular/common';
9  import { ReactiveFormsModule, FormBuilder, FormGroup } from '@angular/forms';
10 import { UbicacionComponent } from '../ubicacion/ubicacion.component';
11 import { EventosDestacadosComponent } from '../eventos-destacados/eventos-destacados.component';
12 import { UserService } from '../../services/user.service';
13 import { UserListComponent } from '../user-list/user-list.component';
14
15 @Component({
16   selector: 'app-perfil',
17   standalone: true,
18   imports: [
19     CommonModule,
20     RouterModule,
21     MatSidenavModule,
22     MatButtonModule,
23     MatFormFieldModule,
24     MatInputModule,
25     NgIf,
26     ReactiveFormsModule,
27     UbicacionComponent,
28     EventosDestacadosComponent,
29     UserListComponent
30   ]
31 })
32 export class PerfilComponent {
33   constructor(private fb: FormBuilder, private userService: UserService, private router: Router) {
34     this.fullName = [''],
35     this.municipality = [''],
36     this.street = [''],
37     this.neighborhood = [''],
38     this.interiorNumber = [''],
39     this.exteriorNumber = [''],
40     this.postalCode = [''],
41     this.location = this.fb.group({
42       lat: [0],
43       lng: [0]
44     });
45   }
46
47   setActiveSection(section: string) {
48     this.activeSection = section;
49     if (section === 'logout') {
50       this.router.navigate(['/login']);
51     }
52   }
53
54   saveUser() {
55     this.userService.createUser(this.userForm.value).subscribe(response => {
56       console.log('User created:', response);
57       alert('User created successfully');
58     }, error => {
59       console.error('Error creating user:', error);
60     });
61   }
62 }
```

```
TS perfil.component.ts U x
src > app > views > pages > perfil > TS perfil.component.ts > PerfilComponent
34 export class PerfilComponent {
35   constructor(private fb: FormBuilder, private userService: UserService, private router: Router) {
36     this.fullName = [''],
37     this.municipality = [''],
38     this.street = [''],
39     this.neighborhood = [''],
40     this.interiorNumber = [''],
41     this.exteriorNumber = [''],
42     this.postalCode = [''],
43     this.location = this.fb.group({
44       lat: [0],
45       lng: [0]
46     });
47   }
48
49   setActiveSection(section: string) {
50     this.activeSection = section;
51     if (section === 'logout') {
52       this.router.navigate(['/login']);
53     }
54   }
55
56   saveUser() {
57     this.userService.createUser(this.userForm.value).subscribe(response => {
58       console.log('User created:', response);
59       alert('User created successfully');
60     }, error => {
61       console.error('Error creating user:', error);
62     });
63   }
64 }
```

Perfil.component.html

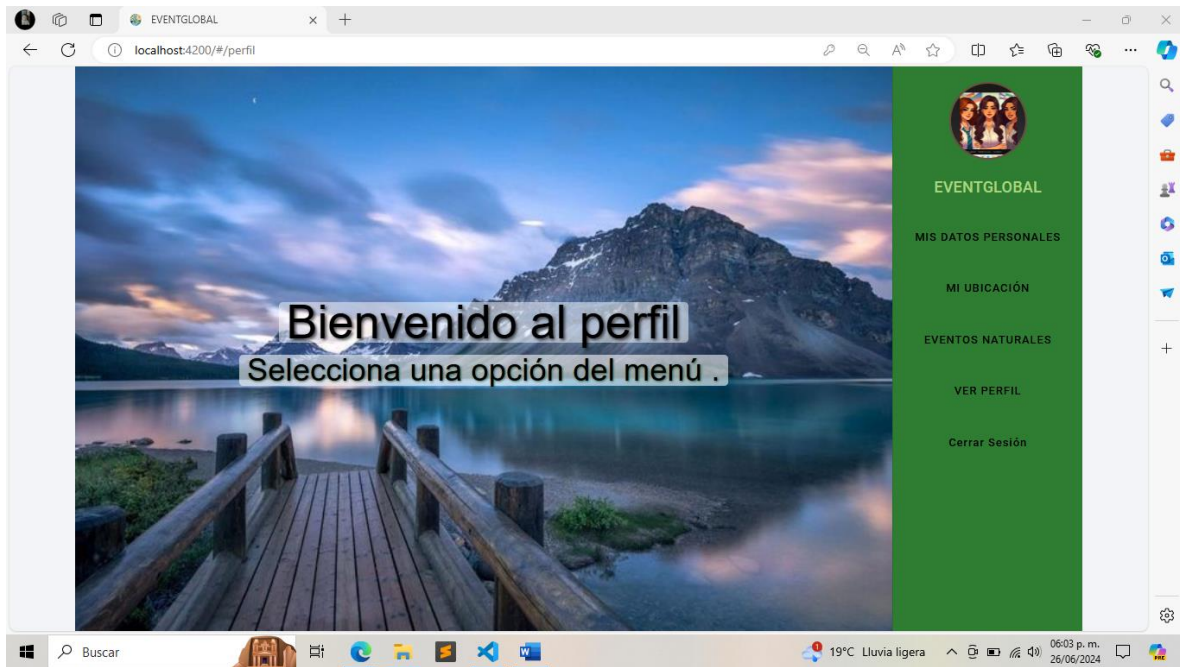
El archivo perfil.component.html define la estructura de la interfaz del componente de perfil en una aplicación Angular. Incluye un sidenav (barra de navegación lateral) que permite a los usuarios navegar entre diferentes secciones como "Mis Datos Personales", "Mi Ubicación", "Eventos Naturales" y "Cerrar Sesión". La plantilla contiene formularios para capturar y mostrar la información del usuario, y utiliza componentes como UbicacionComponent para mostrar la ubicación del usuario en un mapa, y EventosDestacadosComponent para listar eventos importantes. La interfaz es interactiva y facilita la gestión de la información del usuario.

22

```
perfil.component.html U x
src > app > views > pages > perfil > perfil.component.html > div.container > mat-sidenav-container.sidenav-container > mat-sidenav-content > div.ratio > app-ubicac
1 <div class="container">
2   <mat-sidenav-container class="sidenav-container">
3     <mat-sidenav mode="side" position="end" opened class="mat-sidenav">
4       <div class="profile-container">
5         
6         <h2 class="profile-name">EVENTGLOBAL</h2>
7         <div class="buttons-container">
8           <button mat-button (click)="setActiveSection('misdatospersonales')">MIS DATOS PERSONALES</button>
9           <button mat-button (click)="setActiveSection('ubicacion')">MI UBICACIÓN</button>
10          <button mat-button (click)="setActiveSection('eventosnaturales')">EVENTOS NATURALES</button>
11          <button mat-button (click)="setActiveSection('userlist')">VER PERFIL</button>
12          <button mat-button (click)="setActiveSection('logout')">Cerrar Sesión</button>
13        </div>
14      </div>
15    </mat-sidenav>
16    <mat-sidenav-content>
17      <div *ngIf="activeSection === 'ubicacion'" class="ratio">
18        <app-ubicacion</app-ubicacion>
19      </div>
20
21      <div *ngIf="activeSection === 'eventosnaturales'">
22        <app-eventos-destacados</app-eventos-destacados>
23      </div>
24      <div *ngIf="activeSection === 'userlist'">
25        <app-user-list</app-user-list>
26      </div>
27      <div *ngIf="activeSection === 'misdatospersonales'">
28        <h1 class="form-title">COLOCA TUS DATOS COMPLETOS</h1>
29        <form [formGroup]="userForm" class="form-container">
```

```
perfil.component.html U x
src > app > views > pages > perfil > perfil.component.html > div.container > mat-sidenav-container.sidenav-container > mat-sidenav-content > div.ratio > app-ubicac
1 <div class="container">
2   <mat-sidenav-container class="sidenav-container">
3     <mat-sidenav-content>
4       <div *ngIf="activeSection === 'misdatospersonales'">
5         <form [formGroup]="userForm" class="form-container">
6           <div>
7             <mat-label>Número Interior</mat-label>
8             <input matInput placeholder="Número Interior" formControlName="interiorNumber">
9           </mat-form-field>
10          <mat-form-field appearance="fill" class="full-width">
11            <mat-label>Número Exterior</mat-label>
12            <input matInput placeholder="Número Exterior" formControlName="exteriorNumber">
13          </mat-form-field>
14          <mat-form-field appearance="fill" class="full-width">
15            <mat-label>Código Postal</mat-label>
16            <input matInput placeholder="Código Postal" formControlName="postalCode">
17          </mat-form-field>
18          <button mat-raised-button color="primary" type="button" (click)="saveUser()">Guardar</button>
19        </div>
20      </div>
21
22      <div *ngIf="activeSection === ''" class="welcome-section">
23        <h1>Bienvenido al perfil</h1>
24        <p>Selecciona una opción del menú .</p>
25      </div>
26    </mat-sidenav-content>
27  </mat-sidenav-container>
28</div>
```

Resultados de perfil



23

La imagen muestra la página de perfil de la aplicación "EventGlobal". Esta interfaz frontend presenta un fondo visual atractivo con un mensaje de bienvenida "Bienvenido al perfil" y la instrucción "Selecciona una opción del menú". A la derecha, se encuentra un menú de navegación vertical que permite al usuario acceder a diferentes secciones: "Mis Datos Personales", "Mi Ubicación", "Eventos Naturales", "Ver Perfil" y "Cerrar Sesión". Esta estructura facilita la navegación y gestión de la información del usuario dentro de la aplicación.

BAKEND

En ésta sección importa lo que es express, mongoose, body-parser para ser utilizados en el servidor, después con const port, quiere decir que utilizarán el puerto 3000 y mostrará los datos en formato json. Posteriormente la última línea será para conectar la base de datos a mongodb.

```

1  const express = require('express');
2  const mongoose = require('mongoose');
3  const bodyParser = require('body-parser');
4  const cors = require('cors');
5  require('dotenv').config();
6
7  const app = express();
8  const port = 3000;
9
10 // Middleware
11 app.use(bodyParser.json());
12 app.use(cors());
13
14 // Conectar a MongoDB Atlas
15 mongoose.connect(process.env.MONGO_URI).then(() => console.log('MongoDB connected')).catch(err => console.error(err));

```

En esta sección será para definir el esquema con los atributos del usuario en estructura que serán los datos del usuario para ser insertados como el email, password, nombre completo, municipio, ciudad, calle etc.

```

17 // Definir el esquema y modelo de Usuario
18 const userSchema = new mongoose.Schema({
19   email: String,
20   password: String,
21   fullName: String,
22   municipality: String,
23   street: String,
24   neighborhood: String,
25   interiorNumber: String,
26   exteriorNumber: String,
27   postalCode: String,
28   location: {
29     lat: Number,
30     lng: Number
31   }
32 });
33
34 const User = mongoose.model('User', userSchema);
35

```

en esta sección del código serán las funciones para insertar, obtener, editar y eliminar usuario.

1. Esta ruta maneja solicitudes POST a /users.
2. Crea una nueva instancia de User con los datos recibidos en el cuerpo de la solicitud (req.body).
3. Guarda el usuario en la base de datos.
4. Envía el usuario creado como respuesta.
5. Esta ruta maneja solicitudes GET a /users/:id, donde :id es un parámetro de la ruta.
6. Busca un usuario en la base de datos por su ID (req.params.id).
7. Envía el usuario encontrado como respuesta.
8. Esta ruta maneja solicitudes PUT a /users/:id.
9. Actualiza el usuario en la base de datos por su ID (req.params.id) con los datos proporcionados en el cuerpo de la solicitud (req.body).
10. La opción { new: true } indica que la función debe retornar el documento actualizado.
11. Envía el usuario actualizado como respuesta.
12. Esta ruta maneja solicitudes DELETE a /users/:id.
13. Elimina el usuario de la base de datos por su ID (req.params.id).
14. Envía un mensaje confirmando que el usuario ha sido eliminado.

```
36 // Rutas
37 app.post('/users', async(req, res) => {
38   const user = new User(req.body);
39   await user.save();
40   res.send(user);
41 });
42
43 app.get('/users/:id', async(req, res) => {
44   const user = await User.findById(req.params.id);
45   res.send(user);
46 });
47
48 app.put('/users/:id', async(req, res) => {
49   const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true });
50   res.send(user);
51 });
52
53 app.delete('/users/:id', async(req, res) => {
54   await User.findByIdAndDelete(req.params.id);
55   res.send({ message: 'User deleted' });
56 });
57
58 // Nueva ruta para obtener todos los usuarios
59 app.get('/users', async(req, res) => {
60   const users = await User.find();
61   res.send(users);
62 });
63
64 app.listen(port, () => {
65   console.log(`Server running on http://localhost:${port}`);
66 });
```

Ahora en angular se desarrolla la estructura y funcionalidad para el perfil del usuario con el servicio de firebase el cual va a permitir que inicie sesión, se registre o cierre sesión.

```
34     login(email: string, password: string): Promise<void>{
35         return signInWithEmailAndPassword(this.auth, email, password)
36             .then((result) => {
37                 if (!result.user.emailVerified){
38                     sendEmailVerification(result.user);
39                     this.logout();
40                     throw new Error('auth/email-not-verified');
41                 }
42             });
43     }
44
45     logout(){
46         this.auth.signOut();
47     }
48
49     forgotPassword(email: string): Promise<void> {
50         return sendPasswordResetEmail(this.auth, email);
51     }
52
53     isAuthenticated(): boolean {
54         const user = this.auth.currentUser;
55         console.log(user?.uid);
56         console.log(user?.email);
57         return user !== null;
58     }
59
60 }
61
```

en este es un servicio que está diseñado para interactuar con una API REST para realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre recursos de usuario. En la primera parte de las importaciones.

- Injectable: se usa para que Angular sepa que esta clase puede ser inyectada como una dependencia.
- HttpClient: se utiliza para realizar solicitudes HTTP.
- Observable: se utiliza para manejar las respuestas asíncronas de las solicitudes HTTP.

```

TS user.service.ts U X
EventGlobal > src > app > services > TS user.service.ts > UserService
1  import { Injectable } from '@angular/core';
2  import { HttpClient } from '@angular/common/http';
3  import { Observable } from 'rxjs';
4
5  @Injectable({
6    providedIn: 'root'
7  })
8  export class UserService {
9    private apiUrl = 'http://localhost:3000/users';
10
11    constructor(private http: HttpClient) {}
12
13    createUser(user: any): Observable<any> {
14      return this.http.post<any>(this.apiUrl, user);
15    }
16
17    getUser(id: string): Observable<any> {
18      return this.http.get<any>(`${this.apiUrl}/${id}`);
19    }
20
21    updateUser(id: string, user: any): Observable<any> {
22      return this.http.put<any>(`${this.apiUrl}/${id}`, user);
23    }
24
25    deleteUser(id: string): Observable<any> {
26      return this.http.delete<any>(`${this.apiUrl}/${id}`);
27    }
28
29    getUsers(): Observable<any[]> {
30      return this.http.get<any[]>(this.apiUrl);
31    }
32  }

```

En la primera parte del typescript el cual es iniciar sesión maneja la lógica de autenticación y recuperación de contraseña. Aquí está una explicación:

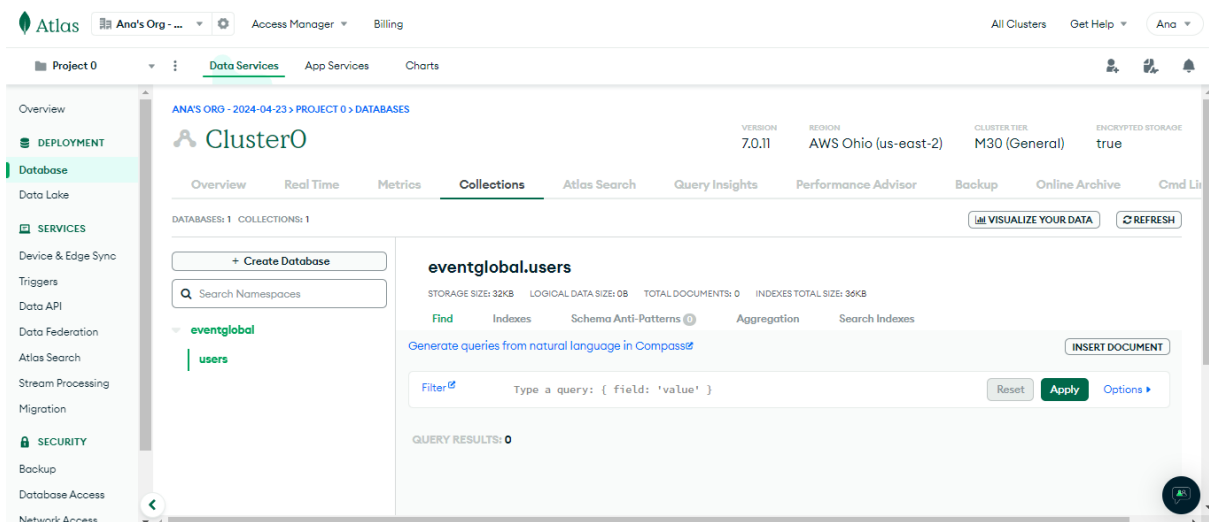
- Component: se usa para definir el componente Angular.
- SharedModule: es un módulo compartido que puede contener componentes, directivas, y pipes reutilizables.
- AuthenticateService: es un servicio para manejar la autenticación de usuarios.
- Router se usa para navegar entre diferentes vistas en la aplicación.
- Define propiedades públicas para manejar el estado del formulario de inicio de sesión y de recuperación de contraseña.

En este archivo, nos permite almacenar una variable de entorno para conectarlo a mongo y sustituir las variables para conectar nuestra base de datos

```
JS server.js  JS server2.js  .env  package-lock.json  JS serveropenai.js  my-projectevent-427101-359186f80dfa.json
.env
1 MONGO_URI=mongodb+srv://root:51e43b1cc8@cluster0.b2pwf.mongodb.net/eventglobal?retryWrites=true&w=majority&appName=Cluster
2
```

29

y podemos ver que en mongo atlas ya está vinculado.



```

TS login.component.ts M X
EventGlobal > src > app > views > pages > login > TS login.component.ts > ...
1  import { Component } from '@angular/core';
2  import { SharedModule } from '../../../shared/shared.module';
3  import { AuthenticateService } from '../../../services/authenticate.service';
4  import { Router } from '@angular/router';
5
6
7  @Component({
8      selector: 'app-login',
9      templateUrl: './login.component.html',
10     styleUrls: ['./login.component.scss'],
11     standalone: true,
12     imports: [SharedModule]
13 })
14 export class LoginComponent {
15     //Para login
16     public email: string = '';
17     public password: string = '';
18     public message: string = '';
19     public type: string = '';
20     public loadinglogin: boolean = false ;
21
22     //Para forgot
23     public visible = false;
24     public emailforgot : string = '';
25     public messagemodal : string = '';
26     public typemodal: string = '';
27     public loadingforgot: boolean = false;
28
29     constructor(private authService: AuthenticateService, private router: Router) { }
30

```

Verifica si los campos de correo electrónico y contraseña están vacíos. Si lo están, muestra un mensaje de error.

Si los campos están llenos, establece loadinglogin en true y llama al método login del servicio de autenticación.

Si la autenticación es exitosa, desactiva loadinglogin y navega al perfil del usuario.

Si hay un error, maneja los diferentes tipos de errores (por ejemplo, correo electrónico no verificado) y actualiza el mensaje y el tipo de mensaje en consecuencia.

```

31 login() {
32   if (this.email === '' || this.password === ''){
33     this.message = "Error: Enter a valid email address or password.";
34     this.type = "danger";
35   } else {
36     this.loadinglogin = true;
37     this.authService.login(this.email, this.password)
38     .then(() => {
39       this.loadinglogin = false;
40       this.router.navigate(['/perfil']);
41     })
42     .catch((error) => {
43       if (error.message === 'auth/email-not-verified'){
44         this.message = "Your email isn't confirmed. Please verify your email to confirm your account";
45         this.type = "warning";
46       } else {
47         this.message = "Error: " + error.message;
48         this.type = "danger";
49       }
50       this.loadinglogin = false;
51     })
52   }
53 }
54

```

Método toggleModal

Este método invierte el valor de la propiedad visible, lo que probablemente se usa para mostrar u ocultar un modal en la interfaz de usuario.

handleLiveDemoChange

cambia el valor de la propiedad visible basándose en un evento recibido. Este evento podría provenir de una interacción del usuario en la interfaz.

forgotpassword

Este método maneja la lógica para la recuperación de contraseñas.

Si el campo de correo electrónico para la recuperación de contraseña (emailforgot) está vacío, establece un mensaje de error y el tipo de mensaje como "danger".

Si el campo no está vacío, establece loadingforgot en true y llama al método forgotPassword del servicio de autenticación (authService).

Si la solicitud de recuperación de contraseña es exitosa, establece un mensaje de éxito y cambia loadingforgot a false.

Si ocurre un error, muestra un mensaje de error con los detalles del mismo y cambia loadingforgot a false.

```

54
55 toggleModal() {
56   this.visible = !this.visible;
57 }
58
59 handleLiveDemoChange(event: any){
60   this.visible = event;
61 }
62
63 forgotpassword() {
64   if (this.emailforgot === '') {
65     this.messagemodal = "Error: Enter a valid email address";
66     this.typemodal = "danger";
67   } else {
68     this.loadingforgot = true;
69     this.authService.forgotPassword(this.emailforgot)
70       .then( () => {
71         this.messagemodal = "Password reset email sent.";
72         this.typemodal = "success";
73         this.loadingforgot = false;
74       })
75       .catch((error) => {
76         this.messagemodal = "Error: " + error.message;
77         this.typemodal = "danger";
78         this.loadingforgot = false;
79       });
80   }
81 }
82
83 }

```

El siguiente componente muestra eventos destacados, como terremotos y datos meteorológicos, utilizando varios servicios.

La primera clase implementa la interfaz OnInit, lo que significa que debe definir el método ngOnInit.

El constructor inyecta tres servicios: EarthquakeService para obtener datos de terremotos, WeatherService para obtener datos meteorológicos y OpenAiService posiblemente para obtener datos o recomendaciones de un modelo de IA.

Llama al método getEarthquakes del servicio EarthquakeService y se suscribe a los datos.

Al recibir los datos, se almacenan en this.earthquakeData.

Mapea this.earthquakeData para crear una lista de marcadores (this.markers) con la posición (latitud y longitud), etiqueta, título, información y opciones de animación para cada evento de terremoto.

Llama al método getWeatherForecast del servicio WeatherService para obtener el pronóstico del tiempo para México durante 3 días y se suscribe a los datos.

Al recibir los datos, se almacenan en `this.weatherData`.

Este componente maneja la lógica de registro de nuevos usuarios.

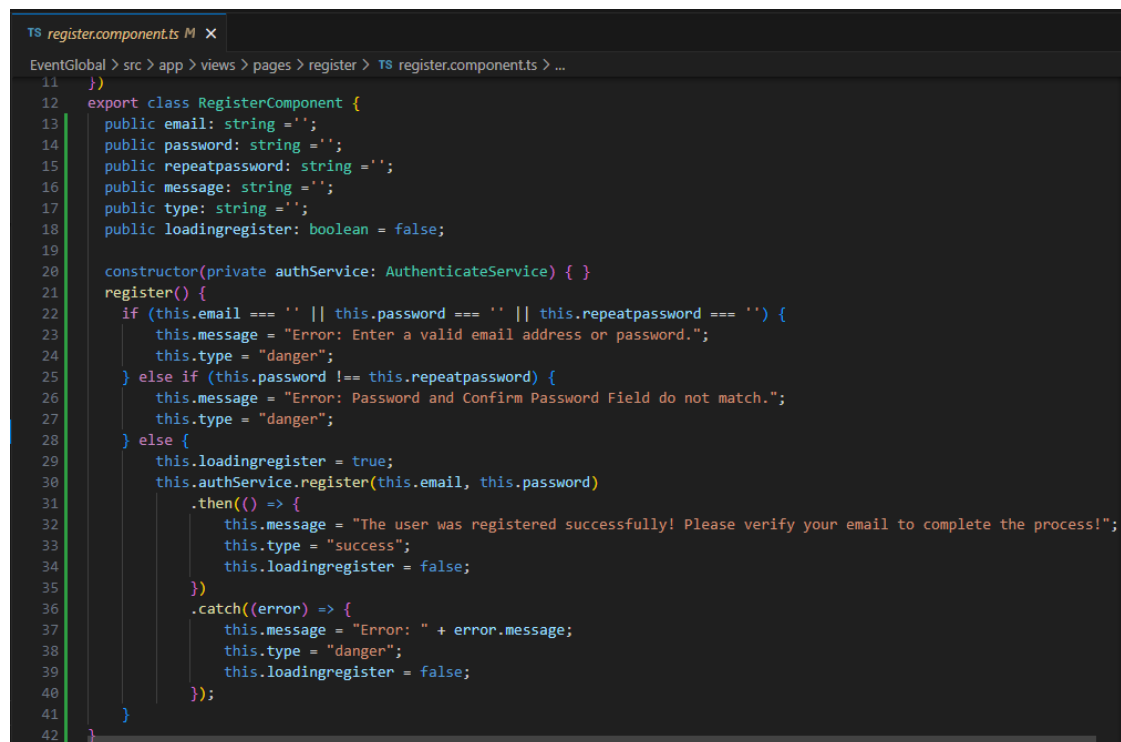
La clase `RegisterComponent` define varias propiedades públicas para almacenar el correo electrónico, la contraseña, la confirmación de la contraseña, mensajes de estado y un indicador de carga. El constructor inyecta `AuthenticateService`, que maneja la lógica de autenticación. Verifica si los campos de correo electrónico, contraseña o confirmación de contraseña están vacíos. Si lo están, muestra un mensaje de error.

Verifica si la contraseña y la confirmación de la contraseña coinciden. Si no coinciden, muestra un mensaje de error.

Si las validaciones anteriores pasan, establece `loadingregister` en `true` y llama al método `register` del servicio de autenticación (`authService`).

Si el registro es exitoso, muestra un mensaje de éxito indicando que el usuario debe verificar su correo electrónico para completar el proceso.

Si ocurre un error durante el registro, muestra un mensaje de error con los detalles del error y desactiva `loadingregister`.



```

11  })
12  export class RegisterComponent {
13    public email: string = '';
14    public password: string = '';
15    public repeatpassword: string = '';
16    public message: string = '';
17    public type: string = '';
18    public loadingregister: boolean = false;
19
20    constructor(private authService: AuthenticateService) { }
21    register() {
22      if (this.email === '' || this.password === '' || this.repeatpassword === '') {
23        this.message = "Error: Enter a valid email address or password.";
24        this.type = "danger";
25      } else if (this.password !== this.repeatpassword) {
26        this.message = "Error: Password and Confirm Password Field do not match.";
27        this.type = "danger";
28      } else {
29        this.loadingregister = true;
30        this.authService.register(this.email, this.password)
31          .then(() => {
32            this.message = "The user was registered successfully! Please verify your email to complete the process!";
33            this.type = "success";
34            this.loadingregister = false;
35          })
36          .catch((error) => {
37            this.message = "Error: " + error.message;
38            this.type = "danger";
39            this.loadingregister = false;
40          });
41      }
42    }

```

En este component muestra la estructura de los datos del usuario que se haya registrado,

```

TS user-list.component.ts U x
EventGlobal > src > app > views > pages > user-list > TS user-list.component.ts > ...
29 export class UserListComponent implements OnInit {
30
31
34   constructor(private userService: UserService, private fb: FormBuilder) {
35     this.userForm = this.fb.group({
36       _id: [''],
37       email: [''],
38       password: [''],
39       fullName: [''],
40       municipality: [''],
41       street: [''],
42       neighborhood: [''],
43       interiorNumber: [''],
44       exteriorNumber: [''],
45       postalCode: [''],
46       location: this.fb.group({
47         lat: [0],
48         lng: [0]
49       })
50     });
51   }
52
53   ngOnInit(): void {
54     this.loadUsers();
55   }
56
57   loadUsers(): void {
58     this.userService getUsers().subscribe((response: any[]) => {
59       this.users = response;
60     });
61   }

```

El constructor inyecta UserService y FormBuilder.

userService se utiliza para interactuar con la API de usuarios.

fb es un servicio que proporciona métodos para crear formularios y sus controles.

userForm es un FormGroup que define la estructura del formulario de usuario, incluyendo campos como _id, email, password, fullName, municipality, street, neighborhood, interiorNumber, exteriorNumber, postalCode, y un FormGroup anidado para location con lat y lng.

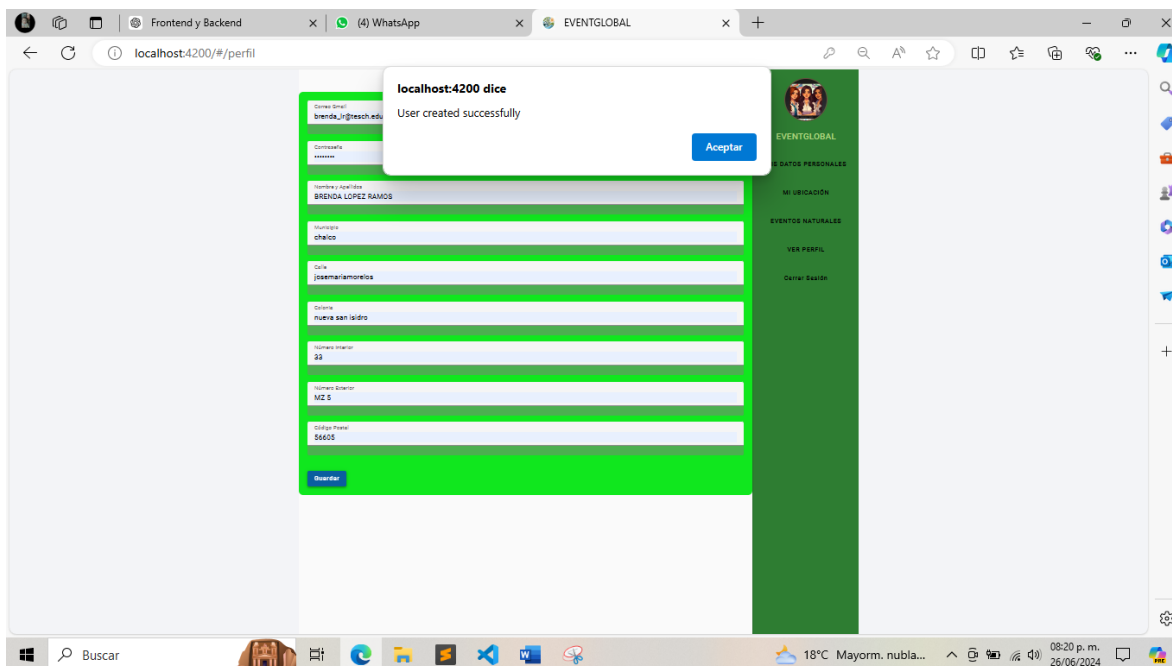
```

TS user-list.component.ts U x
EventGlobal > src > app > views > pages > user-list > TS user-list.component.ts > ...
29 export class UserListComponent implements OnInit {
63   editUser(user: any): void {
64     this.userForm.setValue(user);
65   }
66
67   updateUser(): void {
68     const id = this.userForm.get('_id')?.value;
69     this.userService.updateUser(id, this.userForm.value).subscribe(response => {
70       this.loadUsers(); // Recargar la lista de usuarios
71     });
72   }
73
74   deleteUser(id: string): void {
75     this.userService.deleteUser(id).subscribe(response => {
76       this.loadUsers(); // Recargar la lista de usuarios
77     });
78   }
79 }
80

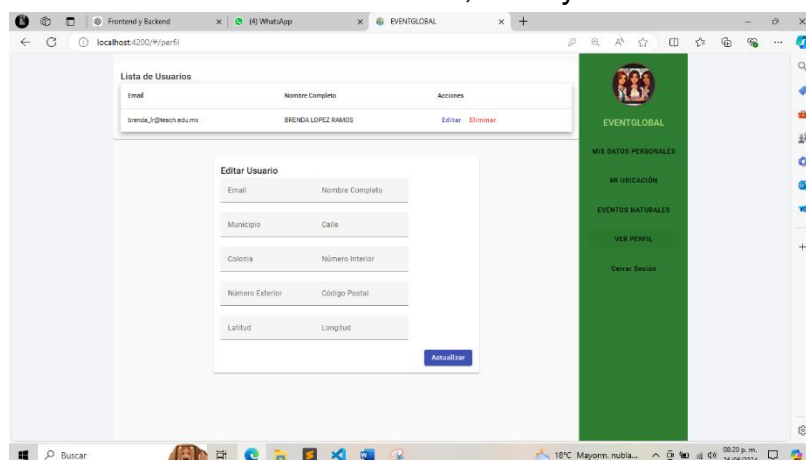
```


RESULTADOS DE BACEND

Para ver los resultados de "EventGlobal", es necesario ejecutar tanto el frontend como el backend. En la imagen, se está utilizando Visual Studio Code para esta tarea. En la sección izquierda, se muestra la estructura de archivos del proyecto frontend de Angular, incluyendo perfil.component.html y otros componentes. Aquí se guardaron los datos y aparece una notificación de que ha sido correcto



En la sección "Ver Perfil" de la aplicación "EventGlobal", se presenta una lista de usuarios con sus respectivos correos electrónicos y nombres completos, además de opciones para editar o eliminar cada usuario. Al seleccionar "Editar", se despliega un formulario que permite modificar detalles del usuario, incluyendo correo electrónico, nombre completo, municipio, calle, colonia, número interior, número exterior, código postal, latitud y longitud. Este formulario facilita la actualización y gestión de la información personal del usuario de manera intuitiva y organizada.



También da la opción de actualizar que como vemos en la imagen ya no dice BRENDA LOPEZ RAMOS sino equipoocho.

Lista de Usuarios

Email	Nombre Completo	Acciones
brenda_lr@tesch.edu.mx	equipoocho	Editar Eliminar

Editar Usuario

Email

brenda_lr@tesch.edu.m

Nombre Completo

equipoocho

Municipio

chalco

Calle

josemariamorelos

Colonia

nueva san isidro

Número Interior

33

Número Exterior

MZ 5

Código Postal

56605

Latitud

Longitud

Actualizar



EVENTGLOBAL

MIS DATOS PERSONALES

MI UBICACIÓN

EVENTOS NATURALES

VER PERFIL

Cerrar Sesión

Y por ultimo elimina, no cerro nuestra sesión por que como estamos ligados al firebase no nos afecta esa parte.

Lista de Usuarios

Email	Nombre Completo	Acciones
-------	-----------------	----------

Editar Usuario

Email

Nombre Completo

Municipio

Calle

Colonia

Número Interior

Número Exterior

Código Postal

Latitud

Longitud

Actualizar



EVENTGLOBAL

MIS DATOS PERSONALES

MI UBICACIÓN

EVENTOS NATURALES

VER PERFIL

Cerrar Sesión

CONCLUSIONES

El proyecto "EventGlobal" es una aplicación web integral diseñada para gestionar información de eventos naturales, datos meteorológicos y perfiles de usuarios, utilizando tecnologías modernas como Angular para el frontend y Node.js para el backend. A lo largo del desarrollo y documentación del proyecto, se han implementado diversas funcionalidades y servicios que hacen de esta aplicación una herramienta robusta y versátil para los usuarios interesados en monitorear y gestionar eventos naturales y sus datos personales.

El primer paso crucial en el desarrollo del proyecto fue la configuración del entorno de desarrollo mediante el archivo ``environment.ts``. Este archivo contiene las claves y configuraciones necesarias para interactuar con varias APIs, como Firebase, Google Maps y OpenAI. La configuración de Firebase es esencial para servicios de autenticación y almacenamiento de datos, mientras que las claves de API de Google Maps y OpenAI permiten integrar mapas y capacidades de inteligencia artificial, respectivamente.

El backend de la aplicación, desarrollado con Node.js, gestiona las operaciones del servidor, incluidas las solicitudes a las APIs y la lógica de negocio. El archivo ``server2.js`` se encarga de levantar el servidor backend, permitiendo la interacción con la base de datos y el manejo de las rutas necesarias para las operaciones CRUD (Create, Read, Update, Delete).

El frontend de la aplicación está construido con Angular, proporcionando una interfaz de usuario dinámica e interactiva. Los componentes clave incluyen ``perfil.component.ts``, ``ubicacion.component.ts`` y ``eventos-destacados.component.ts``, que permiten a los usuarios gestionar su perfil, ver su ubicación en un mapa y consultar eventos naturales destacados, respectivamente.

El componente ``perfil.component.html`` integra un panel de navegación lateral (``sidenav``) que facilita la navegación entre diferentes secciones, como "Mis Datos Personales", "Mi Ubicación", "Eventos Naturales" y "Cerrar Sesión". La estructura del perfil permite a los usuarios visualizar y editar sus datos personales mediante formularios reactivos, integrados con el servicio ``UserService`` para realizar operaciones CRUD sobre los datos del usuario.

La integración de Google Maps se realiza mediante el componente ``ubicacion.component.html``, que muestra un mapa interactivo con un marcador que indica la ubicación del usuario. Esta funcionalidad es crucial para proporcionar una visualización geográfica precisa y relevante, mejorando la experiencia del usuario al permitirle ver y gestionar su ubicación directamente desde la aplicación.

El componente ``eventos-destacados.component.ts`` utiliza la API de EONET (Earth Observatory Natural Event Tracker) para obtener información sobre eventos naturales, como terremotos, incendios y tormentas. Este servicio permite a los usuarios estar informados sobre eventos naturales en tiempo real, proporcionando detalles esenciales como la magnitud, ubicación y fecha del evento.

Una de las características destacadas de la aplicación es el uso de la API de OpenAI para generar recomendaciones relacionadas con eventos naturales. El servicio ``openai.service.ts`` interactúa con el modelo de lenguaje GPT-3.5 de OpenAI, enviando prompts específicos y recibiendo respuestas que se presentan a los usuarios como recomendaciones útiles. Esta funcionalidad añade un valor significativo al proporcionar información basada en inteligencia artificial que puede ayudar a los usuarios a prepararse y responder adecuadamente a los eventos naturales.

El servicio ``weather.service.ts`` se encarga de consumir una API de clima para obtener datos meteorológicos, integrando esta información en la aplicación para proporcionar pronósticos del tiempo y alertas climáticas. Esta funcionalidad es crucial para ofrecer a los usuarios información actualizada sobre las condiciones climáticas, mejorando su capacidad de planificación y respuesta ante situaciones meteorológicas adversas.

La experiencia del usuario es un aspecto central del diseño de "EventGlobal". La interfaz intuitiva y las funcionalidades integradas permiten a los usuarios interactuar de manera fluida con la aplicación, gestionando sus datos personales, visualizando su ubicación y consultando información sobre eventos naturales y condiciones climáticas. El diseño visual, que incluye fondos atractivos y una estructura clara, mejora la usabilidad y hace que la aplicación sea agradable de utilizar.

En resumen, "EventGlobal" es una aplicación web sofisticada y bien estructurada que combina múltiples tecnologías y servicios para ofrecer una herramienta potente y útil para la gestión de eventos naturales y datos personales. La integración de APIs de terceros como Firebase, Google Maps, EONET y OpenAI añade un valor significativo, proporcionando funcionalidades avanzadas y mejorando la experiencia del usuario. A través de un frontend interactivo y un backend robusto, "EventGlobal" demuestra cómo las tecnologías modernas pueden integrarse eficazmente para crear aplicaciones web completas y funcionales. El proceso de desarrollo y documentación detallada asegura que el proyecto no solo cumpla con los requisitos actuales, sino que también esté preparado para futuras expansiones y mejoras.